

RELATÓRIO FINAL - PROJETO SIGN-MAGNITUDE ADDER

Guilherme Pedro da Silva Lima RA: 11202420314

Luís Emílio Gonçalves Mendes RA: 11202421308

Nicholas Fernandez Garcia RA: 11202421369

Pietro Kuszlewicz RA: 11202420430

Resumo

Este relatório descreve o desenvolvimento, simulação e implementação de um somador sinal-magnitude de 4 bits descrito em VHDL, simulado no GTKWave, sintetizado no Quartus Prime Lite e implementado em uma placa FPGA DE10-Lite. O projeto teve como principal objetivo validar o comportamento do circuito, especialmente nos casos de *overflow*. A metodologia adotada envolveu a modelagem do código em VHDL e validação dos resultados utilizando o software de simulação GTKWave. As simulações confirmaram o truncamento nos casos onde houve *overflow*. Após a verificação em software, o sistema foi sintetizado no Quartus Prime Lite e implementado na placa, usando chaves deslizantes e displays de sete segmentos. Os testes práticos confirmaram o comportamento simulado, apresentando plena equivalência e demonstrando o êxito do projeto.

Palavras-chave: VHDL, *Sign-magnitude Adder*, FPGA, DE10-Lite, GTKWave, *Overflow*.

Santo André, SP
Agosto de 2026

1 Introdução

Os sistemas de numeração e a execução de operações aritméticas constituem a base do processamento de dados na eletrônica digital. Neste contexto, utilizou-se o método sinal-magnitude, o qual isola o bit mais significativo (MSB) para indicar o sinal do número (0 para positivo e 1 para negativo), enquanto os bits restantes determinam a sua magnitude.

O presente relatório descreve a adaptação, simulação e implementação de um somador sinal-magnitude de 4 bits (1 bit de sinal e 3 bits de magnitude) desenvolvido em VHDL (*VHSIC Hardware Description Language*). O simulador GTKWave foi utilizado para averiguar o comportamento do algoritmo. Posteriormente, o circuito foi ajustado e mapeado na placa FPGA (*Field Programmable Gate Array*) DE10-Lite, através da ferramenta Quartus Prime Lite, a fim de exibir os sinais, magnitudes e resultado em displays de sete segmentos.

O atual projeto faz-se relevante no aprofundamento prático dos conteúdos abordados na disciplina de Sistemas Digitais, fundamentando-se na obra de Chu [1]. O trabalho sintetiza o desenvolvimento de um somador de sinal-magnitude: desde a descrição comportamental em linguagem VHDL e tratamento dos casos de estouro de *buffer* (*overflow*) até a verificação experimental em FPGA.

2 Objetivos

2.1 Objetivo Geral

Implementar um somador de magnitude com sinal de 4 bits, sendo 1 de sinal e 3 de magnitude, em que o MSB (*Most Significant Bit*) representa o sinal do número (positivo ou negativo) e os bits restantes representam a magnitude do valor.

2.2 Objetivos Específicos

- Implementar o algoritmo somador de magnitude em VHDL.
- Simular o código no GTKWave e verificar seu funcionamento.
- Diagnosticar os casos que estouram o limite do *buffer*.
- Adaptar o código do somador de magnitude para a placa FPGA (*Field Programmable Gate Array*) DE10-Lite no Quartus Prime Lite.
- Adicionar a funcionalidade do visor de sete segmentos, cada dupla representando sinal e magnitude.
- Verificar os resultados obtidos e comparar com os resultados esperados.

3 Justificativa

A realização deste trabalho justifica-se pela contribuição à consolidação do aprendizado teórico e prático sobre sistemas digitais, trabalhando conceitos de representação numérica aplicados a um hardware reconfigurável, da descrição funcional em VHDL até a validação física em FPGA.

Diferentemente de somadores que adotam a representação em complemento de dois, mais comum em aplicações gerais, a abordagem em sinal-magnitude aqui explorada exige um tratamento explícito dos casos de mesmo sinal, sinais opostos e estouro de capacidade, o que a torna pedagogicamente relevante para evidenciar as limitações dessa representação frente às demais. Além disso, a integração do circuito com chaves deslizantes (SW) e displays de sete segmentos na placa permite observar o comportamento do somador em tempo real, contribuindo para a compreensão prática de conceitos que, em geral, são discutidos apenas de forma teórica ou simulada.

4 Metodologia

4.1 Decisões Metodológicas e Modelagem do Circuito

A entrada de dados do sistema foi definida em formato binário utilizando a representação por sinal-magnitude. Utilizam-se 4 bits para representar cada número, sendo 3 bits destinados à magnitude e 1 bit reservado ao sinal. Trata-se de um circuito combinacional, eliminando a necessidade de Máquinas de Estados Finitos (FSM) ou elementos de armazenamento sincronizados por sinal de *clock*.

O circuito foi descrito em linguagem VHDL por meio de uma abordagem funcional, na qual o comportamento lógico do somador de magnitude com sinal foi modelado diretamente com base na estrutura de referência apresentada por Chu [1]. Não foram utilizados diagramas esquemáticos para a entrada do projeto, priorizando a descrição textual em código-fonte.

4.2 Ambiente de Simulação e Verificação Funcional

A validação lógica da arquitetura dividiu-se entre os softwares GHDL (compilação e execução) e GTKWave (visualização dos casos de teste). A simulação adotada foi estritamente funcional, visando comprovar a exatidão das operações lógicas independentemente de atrasos físicos.

Para a validação, desenvolveu-se um arquivo de teste (*testbench*) composto por oito casos:

- Quatro casos de teste operando dentro do limite de representação de 3 bits de magnitude;
- Quatro casos de teste formulados para induzir o estouro de capacidade (*overflow*);

A análise feita através do GTKWave é detalhada na Seção 5.

4.3 Síntese Lógica e Prototipagem em Hardware

Após a validação do funcionamento do sistema, realizou-se a síntese lógica do projeto utilizando a ferramenta Quartus Prime Lite. Para a inserção manual dos operandos, foram utilizados chaves SW. Por sua vez, como forma de tornar a compreensão do funcionamento das operações mais intuitiva, foi adicionada a função que permite a visualização no display de sete segmentos da FPGA. Dentre os seis visores disponíveis, foram divididos em duplas, sendo que cada dupla, um visor é dedicado à exibição do sinal (indicando positivo ou negativo) e o outro exibe o valor numérico da magnitude.

Os testes práticos foram executados manualmente na placa, cobrindo os mesmos cenários simulados no *testbench* para comprovar a equivalência entre a simulação e a aplicação prática, conforme registrado no repositório do projeto [2].

5 Apresentação dos Dados e Análise dos Resultados

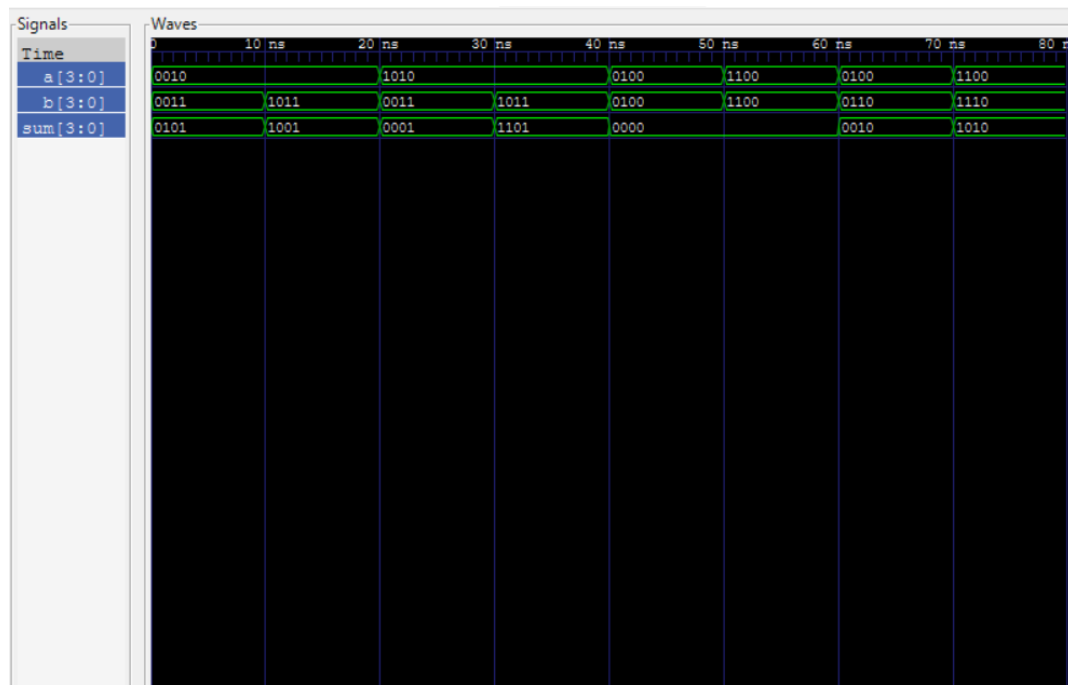


Figura 1: Teste realizado no GTKWave.

Para validar o funcionamento do somador, foram definidos oito casos de teste, sendo quatro dentro do limite de 3 bits de magnitude (variando as combinações de sinal) e quatro que ultrapassam esse limite, forçando o *overflow*, que podem ser visualizados na figura 1:

- **Caso 1:** $2 (0010) + 3 (0011) = 5 (0101)$

- **Caso 2:** $2 (0010) + (-3) (1011) = -1 (1001)$
- **Caso 3:** $-2 (1010) + 3 (0011) = 1 (0001)$
- **Caso 4:** $-2 (1010) + (-3) (1011) = -5 (1101)$
- **Caso 5:** $4 (0100) + 4 (0100) = 0 (0000) - \text{Overflow} (1000 \Rightarrow 000)$
- **Caso 6:** $-4 (1100) + (-4) (1100) = 0 (0000) - \text{Overflow} (1000 \Rightarrow 000)$
- **Caso 7:** $4 (0100) + 6 (0110) = 2 (0010) - \text{Overflow} (1010 \Rightarrow 010)$
- **Caso 8:** $-4 (1100) + (-6) (1110) = -2 (1010) - \text{Overflow} (1010 \Rightarrow 010)$

No caso 6, em particular, o resultado esperado seria (1000), equivalente a menos zero, porém, como o zero não costuma ser associado a um sinal, o código foi adaptado para cobrir esse caso e resultar apenas em zero, sem sinal.

Nos casos de *overflow*, o resultado da soma ultrapassa 3 bits de magnitude. Como o somador manipula apenas 3 bits de magnitude (o quarto bit da entrada é reservado ao sinal), o bit excedente do resultado é descartado, gerando o truncamento observado.

6 Apresentação de um Exemplo de Funcionamento do Programa

Segue um sucinto tutorial para que pessoas com conhecimentos básicos acerca do assunto consigam reproduzir o experimento:

Observação. Os arquivos mencionados abaixo podem ser encontrados no seguinte repositório: https://github.com/Nichsz/Projeto_Sistemas_Digitais

1. Criar um novo projeto no Quartus Prime Lite, selecionando a família MAX 10 FPGA e o dispositivo 10M50DAF484C7G, correspondente à placa DE10-Lite.
2. Adicionar ao projeto o arquivo somador.vhd, contendo a *entity* somador (portas SW_A, SW_B, HEX0 a HEX5), e defini-lo como *Top-Level Entity*.
3. Importar o arquivo de restrições somador_DE10LITE.qsf, que já atribui os pinos físicos da placa a cada entrada e saída: SW_A[3..0] nas chaves SW3–SW0 (pinos C12, D12, C11, C10) e SW_B[3..0] nas chaves SW7–SW4 (pinos A14, A13, B12, A12), além dos seis grupos de pinos HEX0 a HEX5 correspondentes aos displays de 7 segmentos da placa.
4. Compilar o projeto (*Start Compilation*).
5. Na seção *Task*, clicar duas vezes na opção *Program Device (Open Programmer)*, com a DE10-Lite conectada via USB-Blaster e *Mode*: JTAG.

6. Ajustar as chaves SW0–SW3 (operando A) e SW4–SW7 (operando B) em sinal-magnitude, sendo a chave mais significativa de cada grupo (SW3 para A e SW7 para B) responsável por definir o sinal (1 = negativo, 0 = positivo).
7. Observar diretamente, sem necessidade de nenhum botão, os seis displays de 7 segmentos: HEX5/HEX4 exibem o sinal e a magnitude de A; HEX3/HEX2, o sinal e a magnitude de B; e HEX1/HEX0, o sinal e a magnitude do resultado da soma, atualizados em tempo real a cada mudança nas chaves.

Para a simulação funcional prévia, recomenda-se compilar os arquivos com o GHDL (ghdl -a seguido de ghdl -r com geração de arquivo .ghw ou .vcd) e visualizar as formas de onda no GTKWave, aplicando diferentes combinações de sinais e magnitudes nos operandos de teste.

7 Conclusão

Conclui-se que a metodologia de projeto mostrou-se adequada, contribuindo para o cumprimento de todos os objetivos estabelecidos. A implementação do algoritmo somador sinal-magnitude de 4 bits em linguagem VHDL mostrou-se coerente com o modelo teórico. A etapa de simulação funcional no GTKWave permitiu validar a lógica de cálculo para operações com diferentes combinações de sinais, além de diagnosticar com precisão o comportamento do circuito nos casos de *overflow*. A análise das formas de onda confirmou que, ao exceder a capacidade de 3 bits de magnitude, o bit excedente é desconsiderado, mantendo a coerência da representação lógica adotada.

A integração e adaptação do código para a placa FPGA, por meio do Quartus Prime Lite, confirmou a viabilidade da síntese lógica em hardware reconfigurável. A operação das entradas e saídas da placa, utilizando as chaves para inserção dos operandos e os displays de sete segmentos para exibição de sinal e magnitude, ocorreu de maneira adequada. A comparação dos testes físicos com os resultados previstos em simulação demonstrou total equivalência entre os valores esperados e obtidos. Portanto, conclui-se que o projeto cumpriu sua justificativa de realização, consolidando o aprendizado teórico e prático sobre sistemas digitais.

Referências

- [1] Pong P. Chu. *FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version*. Hoboken: John Wiley & Sons, 2008. ISBN: 9780470185315.
- [2] Guilherme Pedro da Silva Lima et al. *Projeto_Sistemas_Digitais*. https://github.com/Nichsz/Projeto_Sistemas_Digitais. Acesso em: 31 jul. 2026.